

08 - Performance y optimización

Ejercicio 08 — Performance y optimización



Foco: performance, optimización de queries, uso del Pool **Tiempo estimado:** 1–2 clases [← Volver al README](#)

Objetivo

Que uses la IA para **encontrar y arreglar problemas de performance**, distinguiendo los que **importan de verdad** de las micro-optimizaciones que no mueven la aguja. Optimizar sin medir es adivinar.

El problema

Algunos puntos del proyecto huelen a problema de performance:

- **SELECT *** en todos lados. ¿Necesitás traer todas las columnas siempre?
 - **No hay paginación:** `GET /api/alumnos` trae **todos** los alumnos. Con 10 está bien, con 100.000 se cae.
 - **Posible N+1:** en `alumnos-service.js`, ¿cómo se resuelve el nombre del curso de cada alumno? Si por cada alumno se hace una query al curso, eso es N+1. Mirá `validarCursoExiste` y pensá qué pasaría si quisieras devolver el nombre del curso junto con cada alumno.
 - **updateAsync hace 2 queries:** primero un `getByIdAsync` y después el `UPDATE`. ¿Se puede en una sola?
 - **El Pool:** ¿está bien configurado? ¿cuántas conexiones? ¿se reusa?
-

Qué tenés que lograr

1. **Medir antes de tocar:** identificá al menos un endpoint y medí cuánto tarda hoy (con muchos datos de prueba — pedíselos a la IA).
 2. Identificar **2 o 3** mejoras reales con la IA.
 3. Implementar **paginación** en el listado (`?page=1&limit=20` o similar).
 4. Implementar al menos **una otra** optimización (columnas explícitas, índice, JOIN en vez de N+1, etc.).
-

Cómo encarar el prompting

💡 La regla de oro: **medir** → **optimizar** → **medir de nuevo**. Sin la medición, no sabés si tu cambio sirvió. Pedile a la IA que te ayude a medir, no solo a cambiar.

Prompt de auditoría:

"Analizá estos repositories buscando problemas de performance que se noten con muchos datos: N+1, falta de paginación, queries innecesarias, `SELECT *`. Ordenalos por impacto real, de mayor a menor, y descartá las micro-optimizaciones que no valen la pena en un CRUD chico."

⚠️ **Trampa #1 — optimización prematura:** la IA te va a sugerir cachés, connection pooling avanzado, y mil cosas. Para un TP, el 90% del valor está en **paginar** y en **no traer columnas de más**. Pedile que priorice por impacto y que te diga cuáles son irrelevantes a esta escala.

⚠️ **Trampa #2 — el índice mágico:** la IA te va a decir "agregá un índice". Un índice acelera lecturas pero **frena escrituras** y ocupa espacio. Preguntale el trade-off y **sobre qué columna** conviene (las que aparecen en `WHERE` y `JOIN`, no porque sí).

⚠️ **Trampa #3 — paginación que rompe el contrato:** si cambiás `GET /api/alumnos` para que pague, **cambiás la respuesta** (ahora devuelve un objeto con metadata, no un array pelado). Eso puede romper a quien consume la API. Decidí concientemente y documentalo.

💡 **Tip de medición:** pedile a la IA un script que inserte 10.000 alumnos de prueba, y medí el endpoint con y sin paginación. La diferencia es tu evidencia.

🔍 Verificación del resultado

- ☐ Tenés un **número antes y un número después** (ms o lo que sea) que demuestra la mejora.
- ☐ La paginación funciona: `?page=2&limit=10` devuelve la página correcta y no se rompe con valores raros (`page=0`, `limit=-5`, `limit=99999`).
- ☐ Si optimizaste un N+1, mostrá que **bajó la cantidad de queries** (contalas con los `console.log` o logs de pg).
- ☐ Las mejoras **no cambiaron el resultado** (los mismos datos, solo más rápido / mejor paginado).
- ☐ Descartaste al menos una sugerencia de la IA por ser irrelevante a esta escala (y lo justificás).

🤔 Pregunta para el oral: *¿cuánto tardaba antes y cuánto ahora? ¿Cómo lo mediste? Si no lo mediste, ¿cómo sabés que mejoró?*

✅ Entrega

Bitácora + commit. **Obligatorio:** incluí las mediciones (antes/después) en la reflexión. Sin números, este ejercicio no está completo.